Redis

Introduction to Redis, In-Memory Data Structure Stores

Core Fundamentals

Understanding In-Memory Speed, Architecture, and Data Types

Why In-Memory? RAM vs DISK

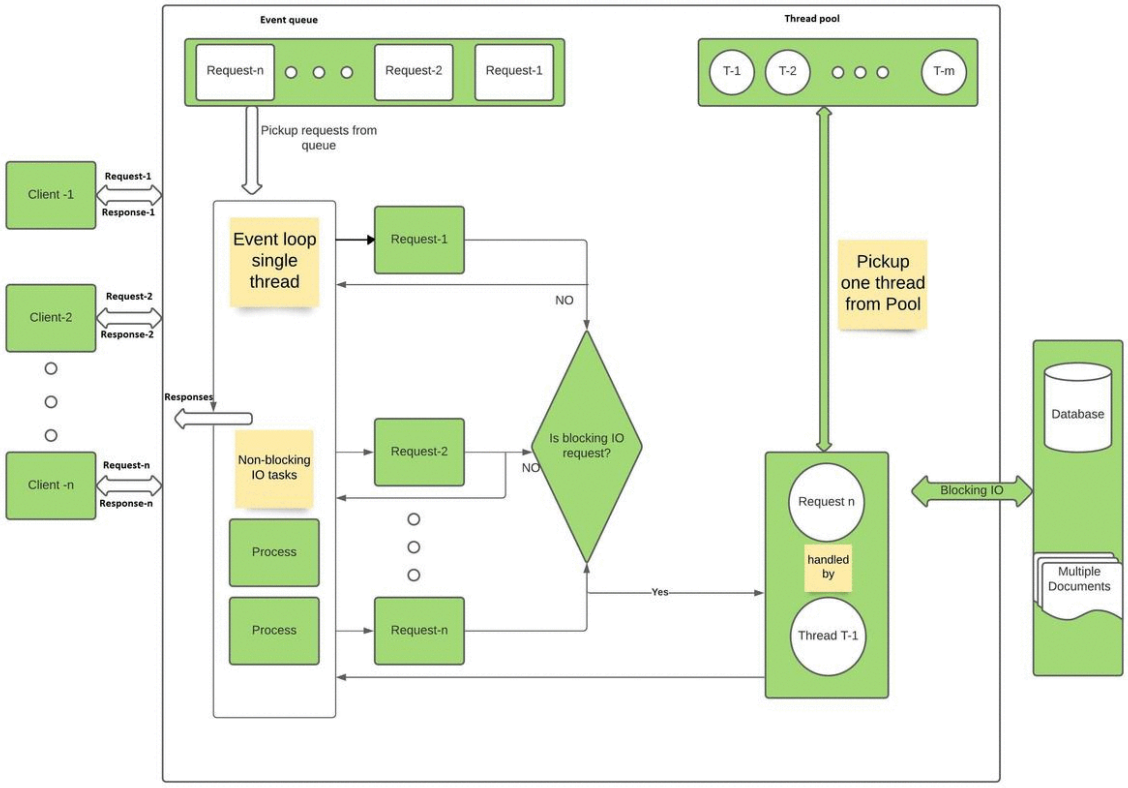
- **Storage Medium** Redis stores active data in Main Memory (RAM), unlike traditional DBs using physical disks (HDD/SSD).
- **Sub-Millisecond Latency** RAM operations execute in microseconds (<1ms), avoiding disk seek time and heavy I/O overhead.
- **High Throughput** Serves hundreds of thousands of concurrent operations per second on modest server hardware.
- **Volatile vs Persistent** Memory is volatile by nature, but Redis provides configurable persistence mechanisms to save data to disk.

Core Data Structure

- **Strings & Hashes**
  - Strings: Text, numbers, binary blobs, atomic counters
  - Hashes: Maps of field-value pairs for object representations
- **Lists & Sets**
  - Lists: Linked lists for message queues and stacks
  - Sets: Unordered unique collections with Set operations (Union, Intersect)
- **Sorted Sets & Streams**
  - Sorted Sets: Sorted by score for real-time leaderboards
  - Streams: Append-only log structure for event streaming

Single-Thread Architecture

- **Single-Threaded Execution** Core command processing runs on one thread, eliminating race conditions, locks, and context switching.
- **I/O Multiplexing (epoll/kqueue)** Non-blocking system calls allow Redis to monitor thousands of client sockets simultaneously.
- **RAM Bottleneck Shift** Memory operations complete so fast that CPU is rarely the bottleneck—network I/O or bandwidth limits occur first.



High Performance Benchmarks

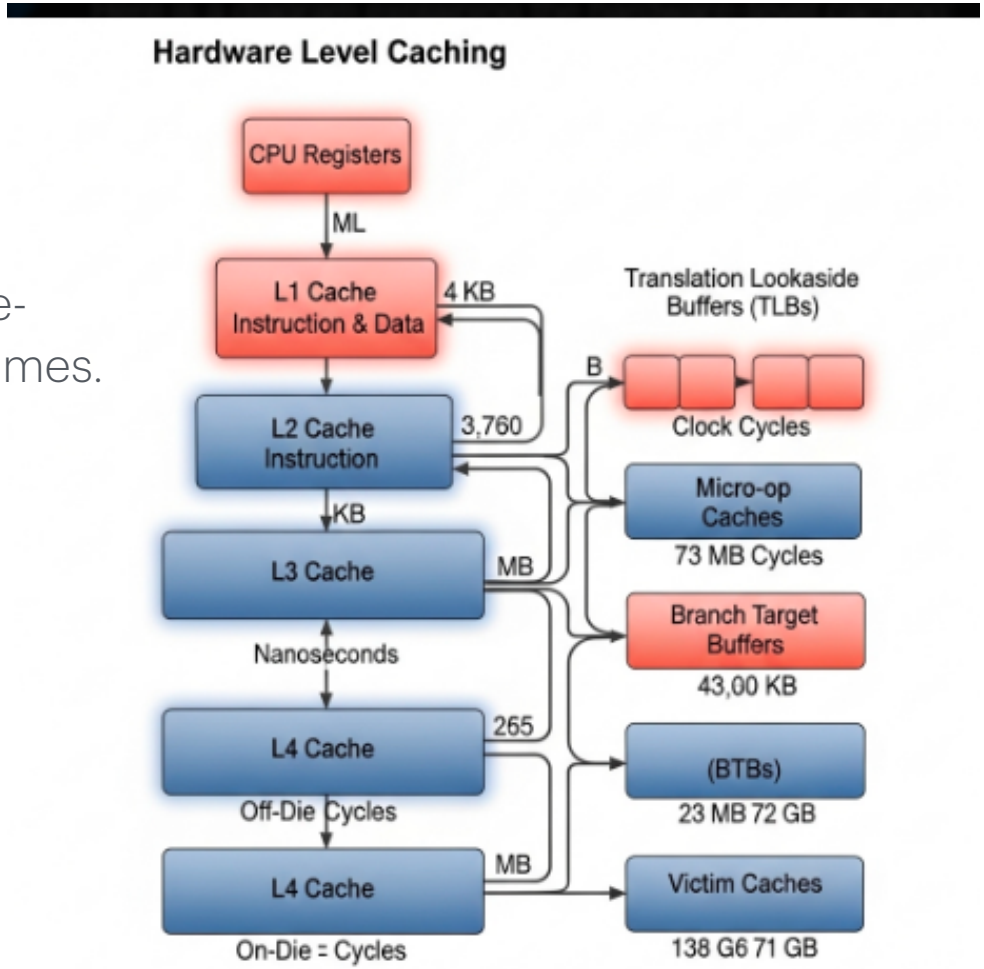
- **100K+** Operations / Second
- **System Performance Impact**
  - Delivers over 100,000 read/write operations per second on a single CPU core.
  - Dramatically reduces primary database CPU and memory load when used as a **cache layer**.
  - Provides predictable microsecond-level response times even under heavy traffic spikes.

Engineering Use Cases

Real-World Architecture & Deployment Patterns

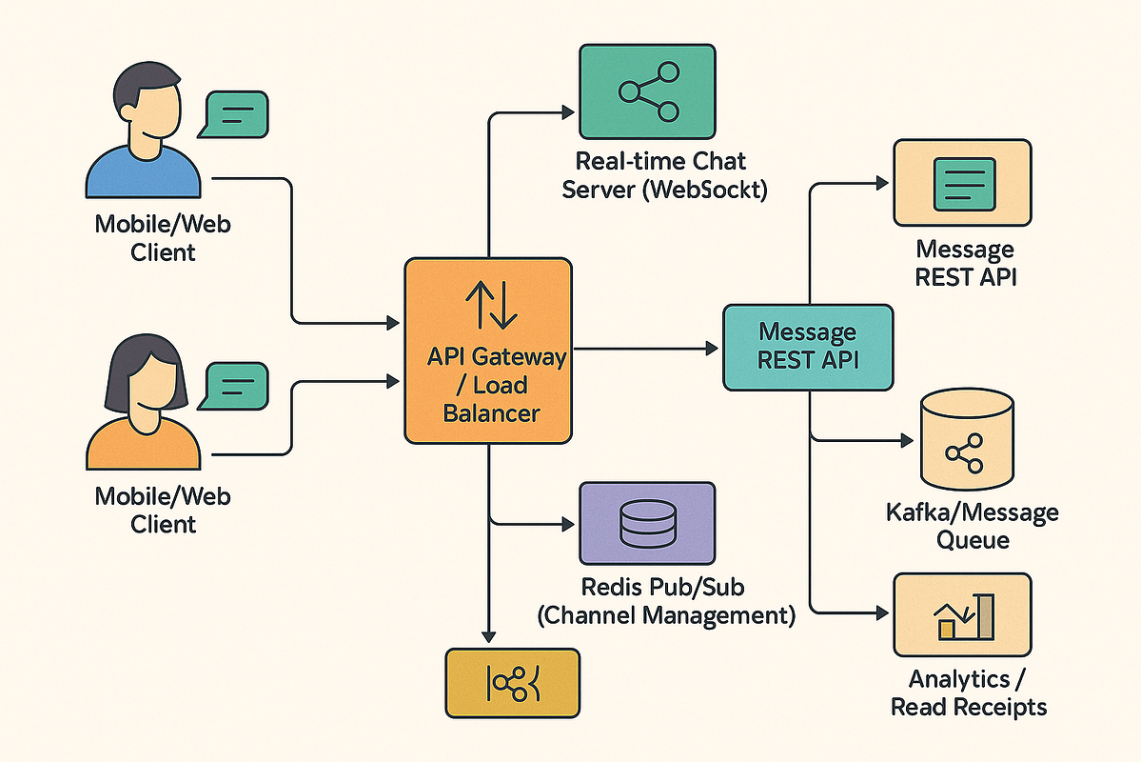
Common Production Use Cases

- Caching Layer
  - Stores hot data using Cache-Aside or Write-Through patterns to accelerate response times.



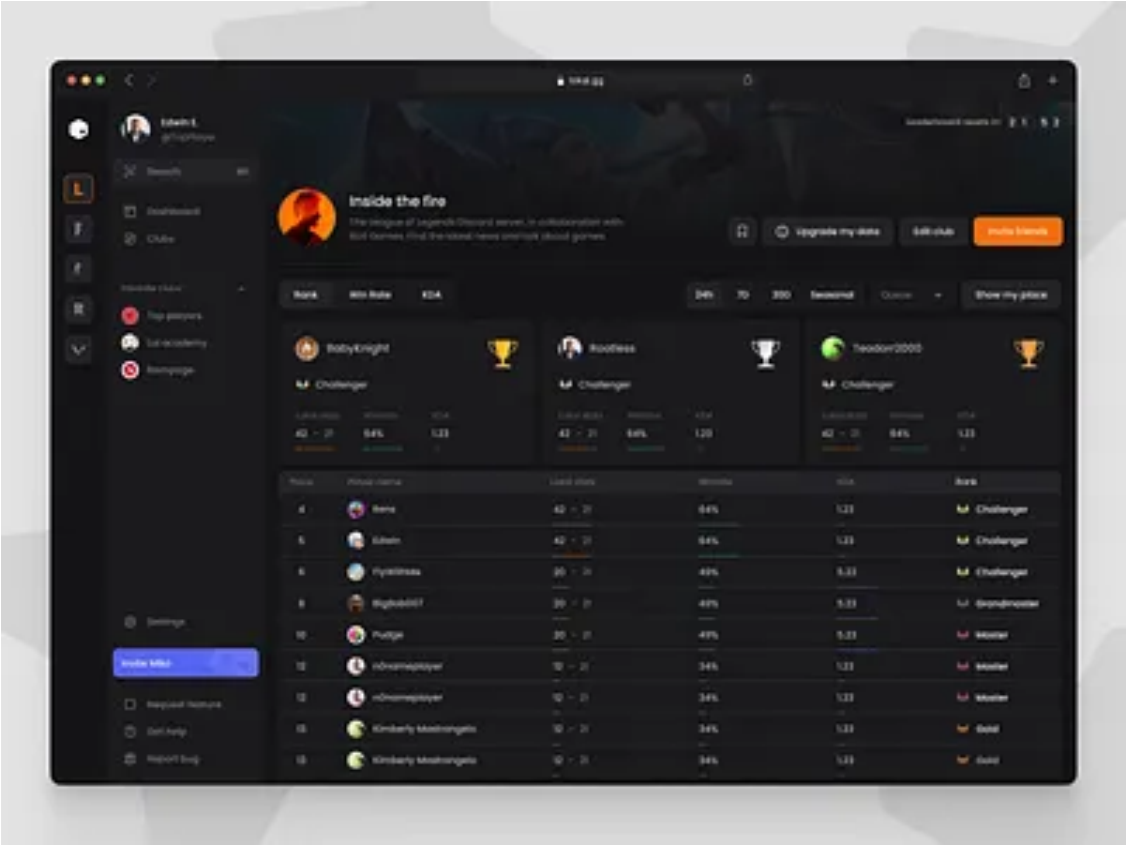
Common Production Use Cases

- Pub/Sub & Queues
  - Decouples microservices using asynchronous Pub/Sub channels and Redis Stream consumers.



Common Production Use Cases

- Rate Limiting
  - Implements sliding window algorithms for API gateway protection and live leaderboards.



Data Persistence Options

- RDB vs AOF Snapshotting
  - **RDB (Redis Database)** Creates point-in-time compact binary snapshots. Fast recovery and minimal runtime performance impact.
  - **AOF (Append-Only File)** Logs every write operation sequentially. Guarantees maximum data durability with configurable sync intervals.

Redis as a Messaging Engine

Pub/Sub, Streams & Real-time Data Pipelines

Why Redis for Messaging?

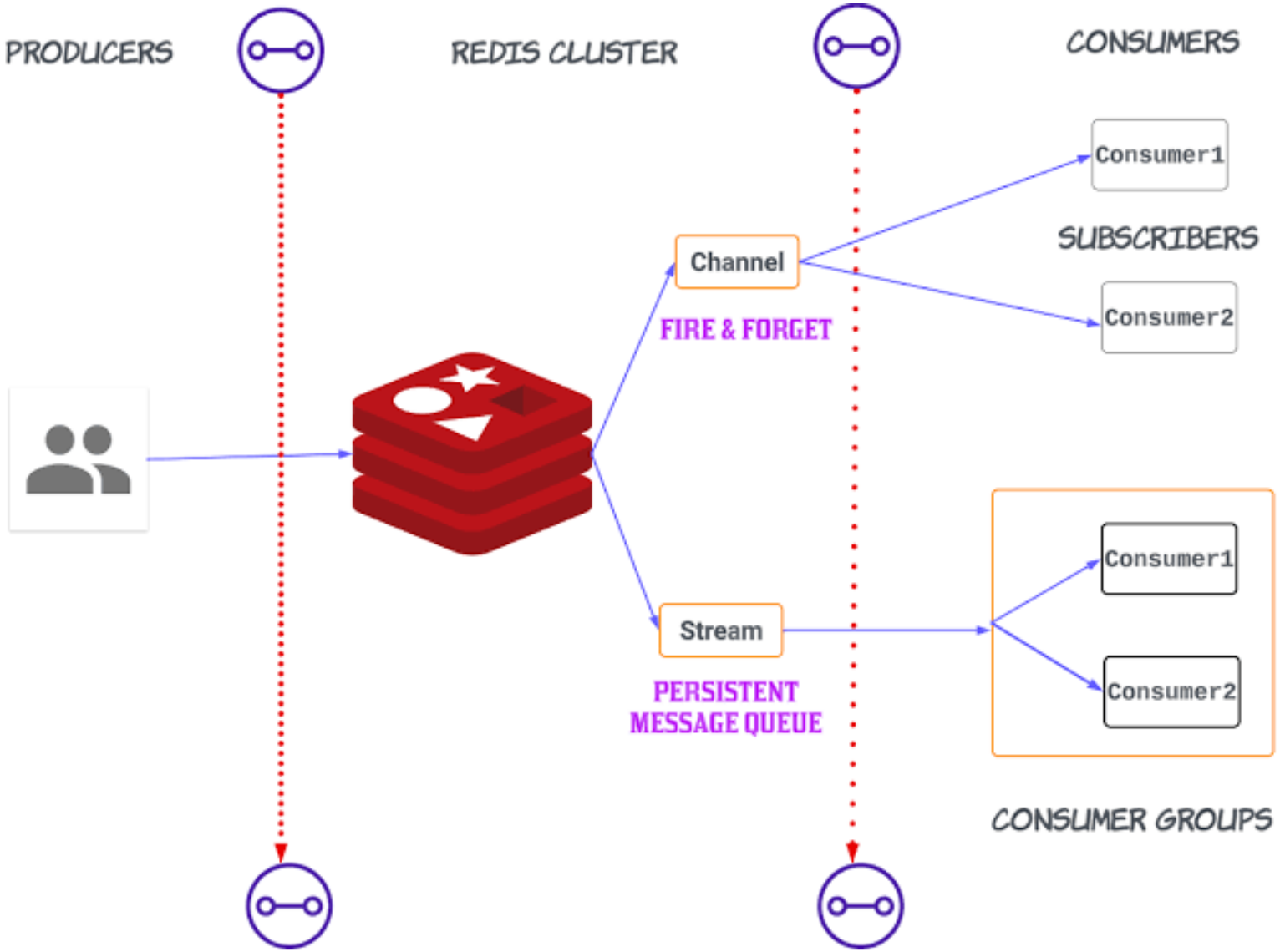
- **Ultra-Low Latency** In-memory architecture ensures sub-millisecond delivery.
- **Simplicity & Speed** Minimal setup, high throughput (100k+ ops/sec).
- **Multiple Patterns Supported**
  - Fire-and-forget (Pub/Sub)
  - Persistent Event Streams (Redis Streams)
  - Message Queues (Lists with LPUSH / BRPOP)
- **Unified Infrastructure** Reuse existing Redis clusters for both caching and messaging.

Pattern 1 - Redis Pub/Sub

- Publish-Subscribe Pattern (**At-most-once delivery**).
- Publishers send messages **without** knowing who the Subscribers are.
- Key Commands:
  - PUBLISH channel message
  - SUBSCRIBE channel
  - PSUBSCRIBE pattern* (Pattern matching)

Fire-and-Forget

- If a subscriber is offline, the message is lost forever.
- No message persistence or history.

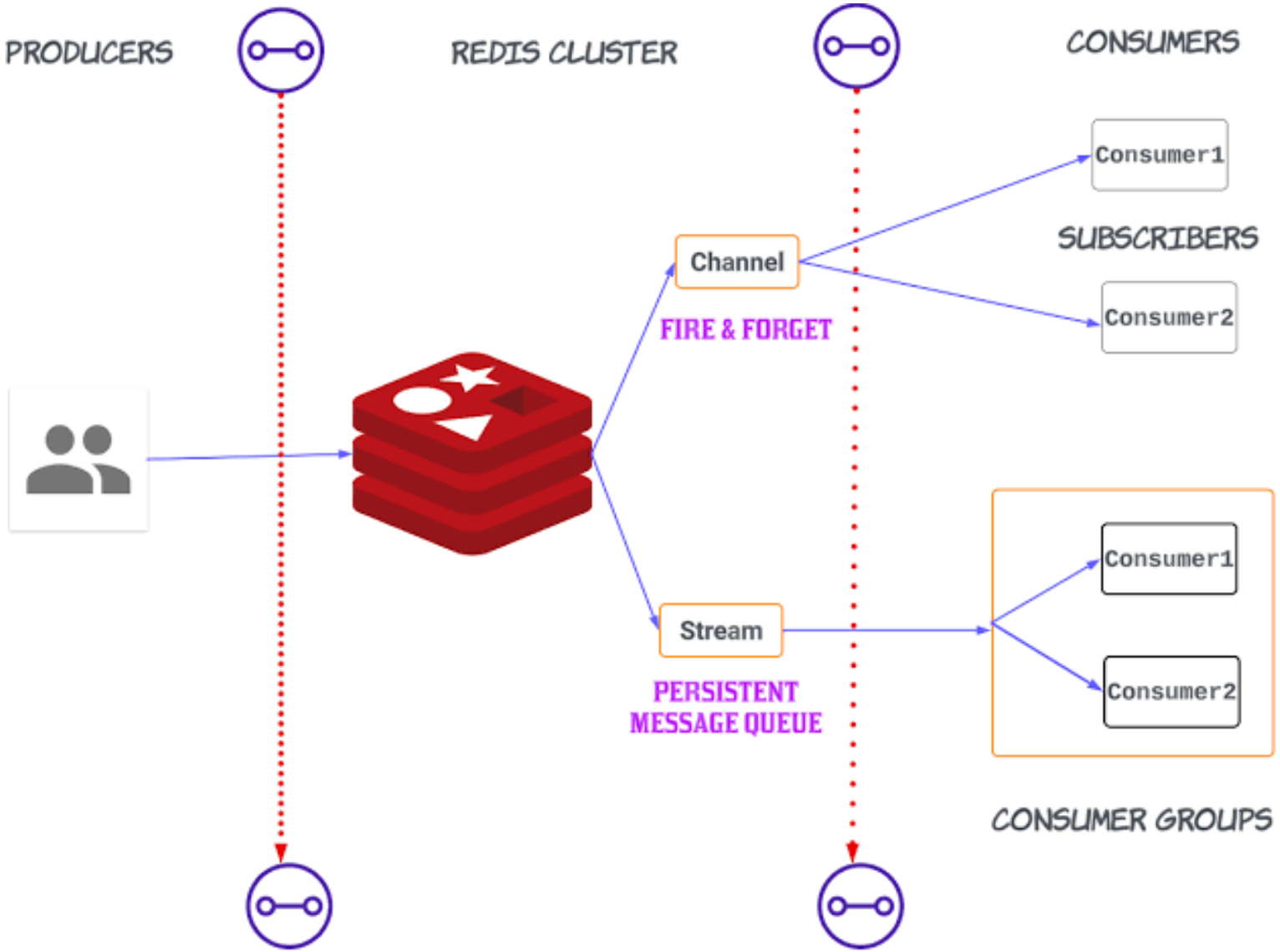

PRODUCERS
REDIS CLUSTER
CONSUMERS
Consumer1
SUBSCRIBERS
Channel
Consumer2
FIRE & FORGET
Consumer1
Stream
Consumer2
PERSISTENT
MESSAGE QUEUE
CONSUMER GROUPS

Pattern 2 - Redis Streams

- **Append-only log** data structure (Introduced in Redis 5.0).
- Key Features:
  - **Persistence** Messages are stored in memory and can be saved to disk.
  - **Unique IDs** Automatically generated IDs (e.g., 1690000000000-0 = timestamp-sequence).
  - **Replayability** Consumers can read old messages starting from any point in time.

Advanced Streams - Consumer Groups

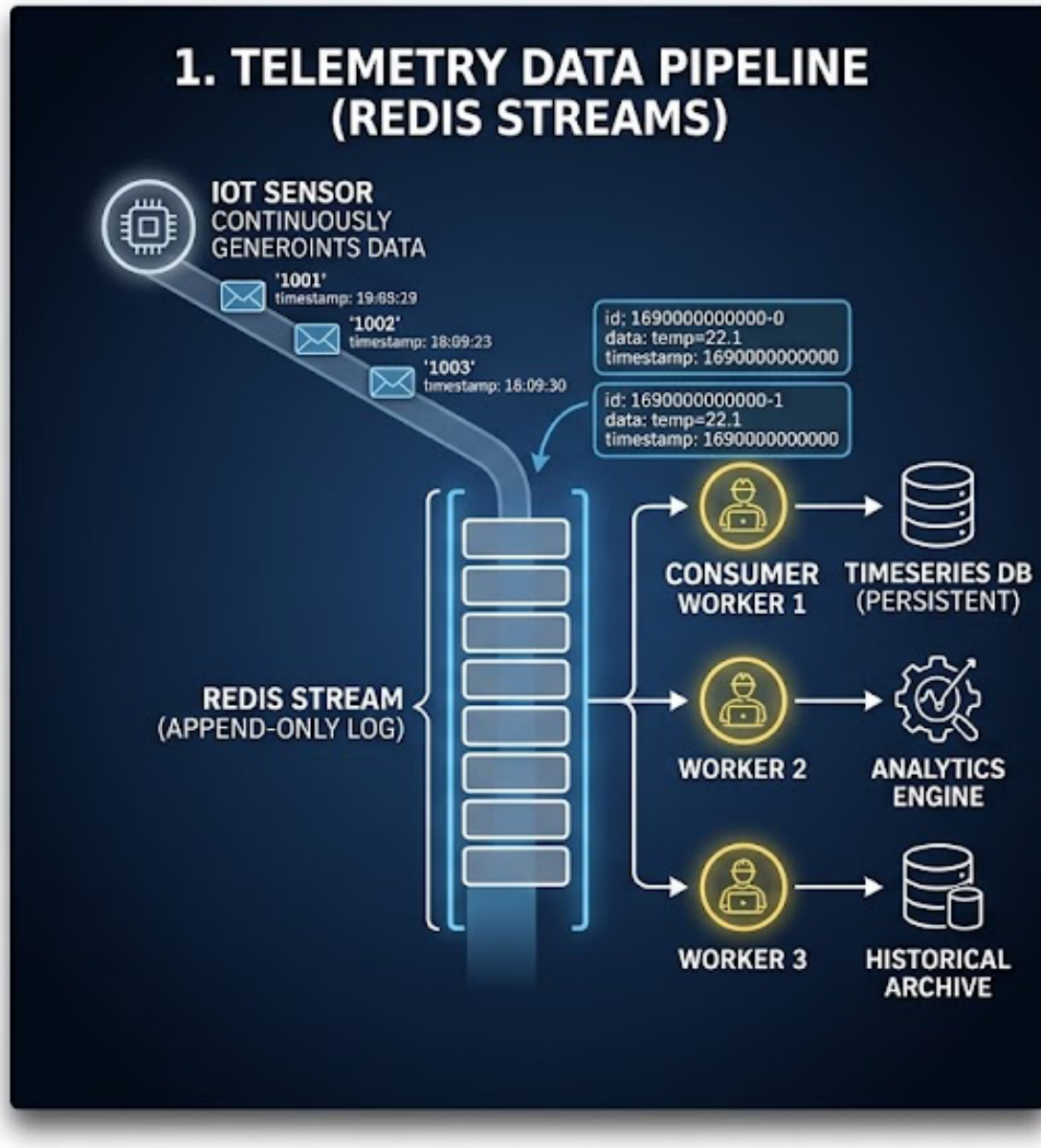
- **Multiple consumers** co-operate to consume a stream.
- **Load Balancing** Different messages are delivered to different consumers in the group.
- **At-Least-Once Delivery** Messages must be acknowledged (XACK).
- **Pending Entries List (PEL)** Tracks unacknowledged messages to handle consumer crashes.


PRODUCERS
REDIS CLUSTER
CONSUMERS
Consumer1
SUBSCRIBERS
Channel
Consumer2
FIRE & FORGET
Consumer1
Stream
Consumer2
PERSISTENT
MESSAGE QUEUE
CONSUMER GROUPS

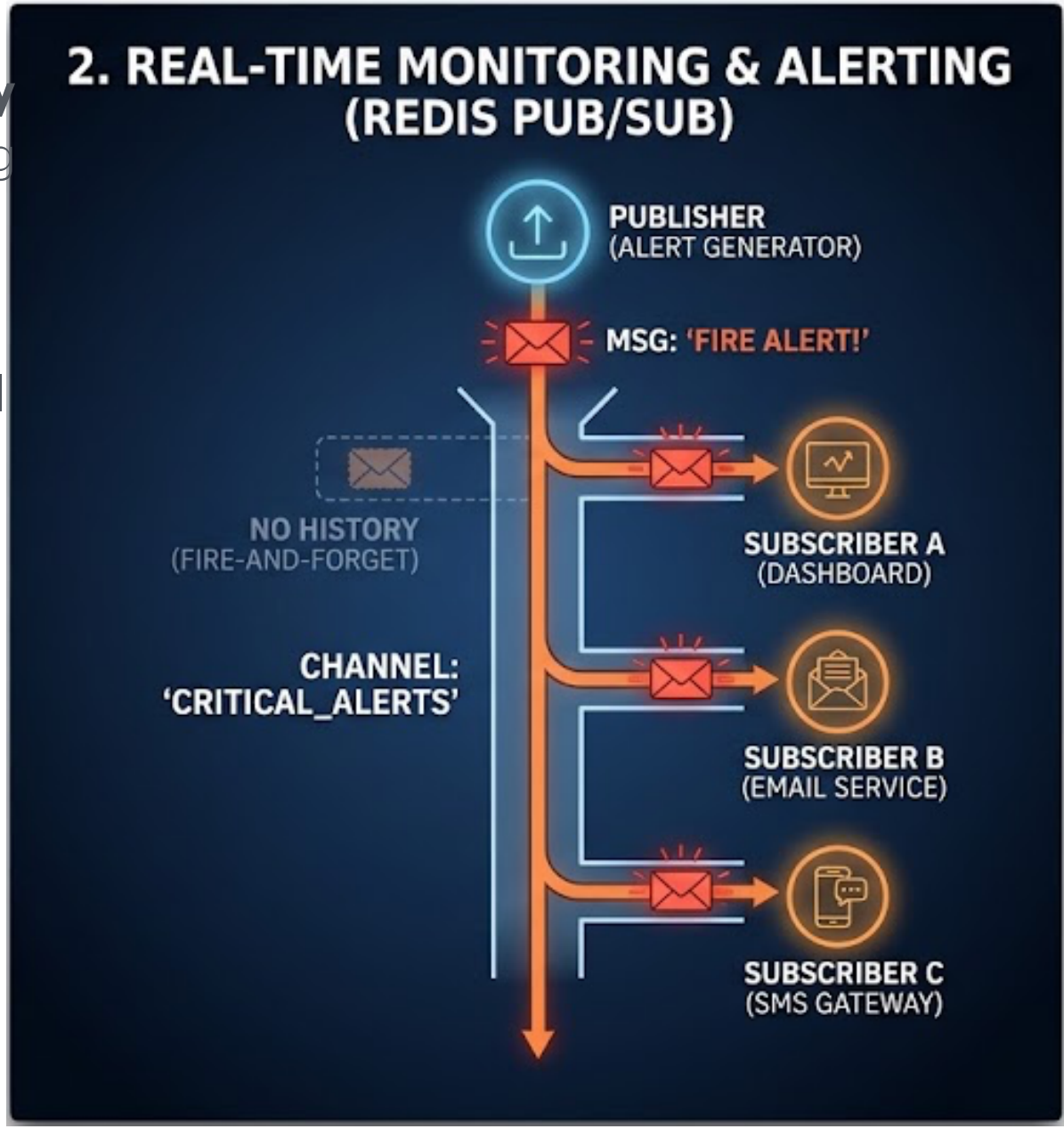
Real-World Use Cases

- Use Case 1: **Real-time Dashboards & Chat (Pub/Sub)**
  - Instant WebSockets broadcasting, live audio/video chat signaling.
- Use Case 2: **Order Processing Pipeline (Redis Streams)**
  - E-commerce checkout: Order Service -> Stream -> Payment Service / Inventory Service.
- Use Case 3: **Rate Limiting & Event Sourcing (Streams)**
  - Tracking user activity events with timestamps for audit logs.

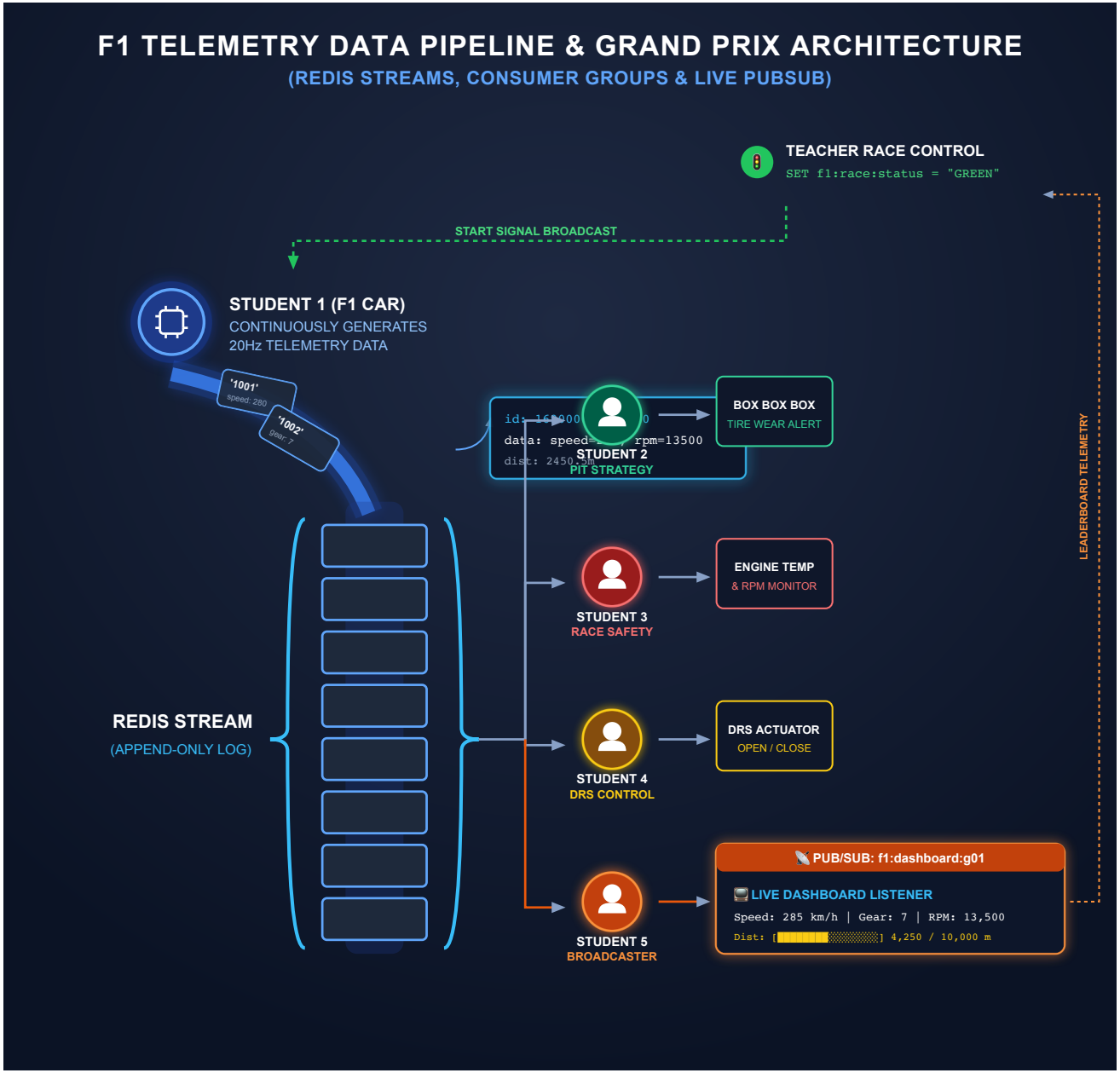
- Data is stored as an immutable, **ordered sequence** of **time-series** entries with auto-generated unique IDs.
- Messages are saved in **memory** and **disk**, allowing consumers to **read past records or replay missed** events anytime
- Acts as a **buffer** between **high-frequency** IoT sensors and **backend** databases to prevent system overload.
- Enables **multiple worker** nodes to share the processing workload concurrently with message acknowledgment (XACK) support.



- Messages are **pushed instantaneously** to connected subscribers **without** being **saved** in memory or disk.
- Delivers **ultra-low latency broadcasting** ideal for real-time critical event notifications.
- **A single publish** command immediately **dispatches** alert events across **all active listening channels** and services.
- If a subscriber service is **offline** during publication, it will **not receive** the missed message.



F1 Telemetry Processing Pipeline & F1 Grand Prix Race Control


F1 TELEMETRY DATA PIPELINE & GRAND PRIX ARCHITECTURE
(REDIS STREAMS, CONSUMER GROUPS & LIVE PUBSUB)
TEACHER RACE CONTROL
SET f1:race:status = "GREEN"
START SIGNAL BROADCAST
STUDENT 1 (F1 CAR)
CONTINUOUSLY GENERATES
20Hz TELEMETRY DATA
'1001'
speed: 280
'1002'
gear: 7
REDIS STREAM
(APPEND-ONLY LOG)
data: speed= rpm=13500
dist: 2450.5m
STUDENT 2
PIT STRATEGY
BOX BOX BOX
TIRE WEAR ALERT
STUDENT 3
RACE SAFETY
ENGINE TEMP
& RPM MONITOR
STUDENT 4
DRS CONTROL
DRS ACTUATOR
OPEN / CLOSE
STUDENT 5
BROADCASTER
PUB/SUB: f1:dashboard:g01
LIVE DASHBOARD LISTENER
Speed: 285 km/h | Gear: 7 | RPM: 13,500
Dist: [ ] 4,250 / 10,000 m
LEADERBOARD TELEMETRY